

Small Language Models (SLMs)

One-page summary of the key ideas, demo, benchmark, and clickable references.



■ 1. What are SLMs?

- Compact AI language models, often in the few-billion-parameter range.
- Designed to deliver useful language capability with lower compute needs.
- Practical for local, fast, private, and lower-cost AI applications.

■ 2. Why use SLMs?

- Faster inference and lower latency on normal hardware.
- Lower RAM and storage needs compared with large models.
- Better privacy when run locally because prompts do not need to leave the laptop.
- Useful for edge devices, classrooms, prototypes, and small projects.

■ 3. How are strong SLMs built?

- High-quality training data and synthetic instruction data.
- Knowledge distillation: a larger teacher model helps train a smaller student model.
- Quantization: 4-bit or 8-bit compression reduces memory usage.
- Efficient architecture choices improve speed and throughput.

■ 4. Models discussed/tested

- Phi-3: strong small model family from Microsoft.
- Gemma 3: Google model family with efficient local use cases.
- Mistral 7B: efficient open-weight model with strong quality for its size.
- Local Ollama models used for demo and benchmark comparisons.

■ 5. Strengths

- Good for focused everyday tasks: summarization, simple Q&A, drafting, classification.
- Easy to deploy locally with tools like Ollama.
- Lower cost makes repeated testing and student projects more realistic.
- Can be paired with dashboards to show latency, RAM, and output quality.

■ 6. Limitations

- Can hallucinate facts, especially on unknown or future events.
- Usually weaker than larger models in deep reasoning or long-context tasks.
- Performance depends heavily on prompt design and hardware.
- RAM charts may be noisy: negative RAM change means the system freed memory after the run, not that the model used negative RAM.

Key takeaway: SLMs do not replace large models everywhere, but they are a strong choice when speed, cost, local deployment, and privacy matter.

Demo + Benchmark Workflow

How the project measures SLM behavior locally and where the audience can click for references.



Demo workflow

- User prompt is selected from the prompt file.
- Python script sends the prompt to a local Ollama model through the Ollama API.
- The model response is saved into CSV/JSON for repeatable presentation output.
- Streamlit dashboard displays prompt, response, model, and timing information.

Benchmark focus

- Compare models on latency, response quality, tokens/sec, and RAM change.
- Use the same prompt set across models to keep comparison fair.
- Dashboard helps explain trade-offs visually instead of only showing raw code.
- Interpret RAM as approximate net change because laptop background processes can affect readings.

What to say from the results

- Small models can answer many simple tasks quickly and locally.
- Larger or stronger models usually give better responses but need more time/resources.
- The best model depends on the use case: speed, quality, memory, and privacy.
- Benchmarking makes the trade-off visible instead of only theoretical.

Project message

- Our project shows that SLMs are not just theory: they can be run, measured, and explained on a normal student laptop.
- The demo proves local usability; the benchmark proves measurable trade-offs.
- The dashboard makes the result understandable for non-technical classmates.

Clickable reference links

Phi-3 technical report	https://arxiv.org/abs/2404.14219
Gemma 3 technical report	https://arxiv.org/abs/2503.19786
Mistral 7B paper	https://arxiv.org/abs/2310.06825
llama.cpp - local inference toolkit	https://github.com/ggml-org/llama.cpp
bitsandbytes - quantization toolkit	https://github.com/bitsandbytes-foundation/bitsandbytes
Ollama - local model runner	https://ollama.com/
Project GitHub repository	https://github.com/ShoumikDutta/Student-project-for-a-presentation-on-Small-Language-Models

Tip: Open this PDF in a browser or PDF reader and click the blue links above.